

Philosophie des tests et conception de formulaires de demande de test

1. Pourquoi formaliser les demandes de tests ?

Un formulaire de test détaillé n'est pas une contrainte administrative : c'est un **outil de communication universel**. Il sert d'interface entre :

- les développeurs,
- les testeurs (QA),
- les Product Owners,
- les équipes d'intégration,
- et même les outils de tests automatisés.

Chaque IDE ou plateforme possède son propre framework de tests automatisés (ATF pour ServiceNow, JUnit pour Java/Kotlin, XUnit pour C#, PyTest pour Python...).

Mais tous reposent sur les mêmes principes fondamentaux : un objectif clair, des prérequis, un protocole, un résultat attendu, et des conditions d'échec.

Le formulaire devient donc un **langage commun**, indépendant de la technologie, du langage ou de l'outil. Il permet de décrire un test de manière neutre, compréhensible et reproductible, quel que soit l'environnement.

2. Philosophie générale des tests

2.1. Un test doit être reproductible

Un test doit donner le même résultat, exécuté par n'importe qui, à n'importe quel moment.

- Pas de dépendance aux données de production.
- Les données nécessaires doivent être créées dans les étapes du test.
- Les prérequis doivent être explicites.

2.2. Un test doit avoir un objectif clair

Le test doit répondre à une question simple : « **Que suis-je en train de vérifier exactement ?** »

2.3. Un test doit être vérifiable

Le résultat attendu doit être observable, mesurable et non ambigu.

3. Les champs essentiels d'un formulaire de test

3.1. Cas de test (titre descriptif)

Une phrase claire qui résume l'objectif du test. Exemple : « **Un administrateur peut supprimer un utilisateur via le bouton Supprimer** ».

3.2. Conditions préalables

Ce sont les ressources nécessaires, pas des données métier existantes.

Important : Les données métier doivent être créées dans les étapes du test, pas supposées déjà présentes.

3.3. Étapes du cas de test

Le protocole détaillé, étape par étape, permettant de reproduire le test.

3.4. Résultats attendus

Ce qui doit se produire si le test est concluant. Le résultat doit être concret et observable.

3.5. KO (conditions d'échec spécifiques)

Les conditions particulières qui invalident le test, même si l'action principale semble fonctionner.

4. Penser les tests : unitaire, implémentation, E2E

4.1. Tests unitaires

Ils valident une petite unité de code isolée. Rapides, automatisables, essentiels pour la stabilité du code.

4.2. Tests d'implémentation / d'intégration

Ils valident les interactions entre plusieurs composants : API, base de données, services externes...

4.3. Tests E2E (End-to-End)

Ils valident un parcours complet du point de vue de l'utilisateur final.

Le formulaire de test décrit un scénario métier. Ce scénario peut ensuite être automatisé dans n'importe quel framework, car il suit une structure universelle : prérequis → étapes → résultats attendus.

5. Comment choisir le bon type de test ?

- **Logique locale** → test unitaire
- **Interaction entre modules** → test d'implémentation
- **Parcours utilisateur** → test E2E

Un bon système de tests combine ces trois niveaux pour garantir robustesse, cohérence et qualité.

6. Conclusion

Un formulaire de test détaillé est un outil de communication, un support de collaboration et un pont entre les équipes et les outils. Il permet de décrire un test de manière neutre, reproductible et universelle, indépendamment du langage, de la technologie ou du framework utilisé.